

Analysis of scRNA-seq data from glioblastoma (GBM) patients with scBubbletree

Simo Kitanovski (simo.kitanovski@uni-due.de)

Contents

Background	1
Data	1
Data processing	2
Step 1: determining the clustering resolution r for Louvain clustering	3
Step 2-3: clustering and hierarchical bubble grouping $\rightarrow$ bubbletree	5
Step 4: bubbletree visualization and annotation	5
Lets inspect the bubbletree	5
Annotating the bubbletree with cell features	5
What is the cell type composition of the bubbles?	6
What is the tissue composition of the bubbles?	7
Compositional analysis: are cells from different samples evenly mixed within bubbles?	9
Quality control with <i>scBubbletree</i>	10
Other cell features: cell phase and gene expression	12
Summary	16
Session Info	16

Background

Here we will use *scBubbletree* to analyze scRNA-seq data with over 45,000 cells from five human glioblastoma (GBM) patients¹. From each patient, we have paired samples from three tissues: tumor center, peripheral infiltration zone, and blood. This vignette aims to demonstrate that *scBubbletree* can evaluate complex scRNA-seq data from solid tumors. In our analysis, we compare our results with those presented by Schmassmann et al., 2023, and highlight the advantages of using *scBubbletree*.

Data

The publicly available scRNA-seq data² has been generated by 10X Genomics Chromium Single Cell Controller. The raw data (barcode-gene count matrix) is accompanied by supplementary meta-data. The metadata includes the following cell-specific features:

- cell type labels (coarse- and fine-grained)

¹Schmassmann et al., 2023 <https://doi.org/10.7554/eLife.92678.2>

²<https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE197543>

- cell phase labels
- sample name
- tissue site
- quality metrics: % of transcripts from mitochondrial/ribosomal genes

Data processing

Data processing was performed with the R-package *Seurat*. Gene expressions were normalized (function `NormalizeData`), highly-variable genes were identified (function `FindVariableFeatures`), and the expressions were scaled and centered gene-wise (function `ScaleData`). Next, principal component analysis (PCA) was performed with the function `RunPCA` based on the 5,000 most variable genes.

In the data, we saw that the first 15 principal components capture most of the variance in the data (data not shown), and the proportion of variance explained by each subsequent principal component was negligible. Thus, we used the single-cell projections (embeddings) in 15-dimensional feature space, $A^{45,466 \times 15}$.

```
## Description: we used this script to download the raw data and process it
## as explained in the above paragraph .This script can be used to generate
## the data GBM data
##
## create directory
dir.create(path = "case_study/")
##
## download the data from:
## https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE197543
##
## extract data:
gzip -d GSE197543_UMIsMatrix.txt.gz
gzip -d GSE197543_colData.txt.gz
##
## read.csv
u <- read.csv(file = "case_study/GSE197543_UMIsMatrix.txt", sep = "\t")
##
## create Seurat object from the raw counts and append the meta data to it
d <- Seurat::CreateSeuratObject(counts = u, project = 'GBM')
##
## get meta
meta <- read.csv(file = "data/CS_Glio/GSE197543_colData.txt", sep = "\t")
meta_tissue <- do.call(rbind, strsplit(x = rownames(d@meta.data), split = "_"))
d@meta.data <- cbind(d@meta.data, meta)
d@meta.data$tissue <- meta_tissue[,4]
d@meta.data$patient <- meta_tissue[,3]
d@meta.data$cell_id <- 1:nrow(d@meta.data)
##
## normalize data
d <- NormalizeData(object = d, normalization.method = "LogNormalize")
##
d <- FindVariableFeatures(object = d, selection.method="vst", nfeatures=5000)
##
d <- ScaleData(object = d, features = VariableFeatures(object = d))
##
d <- RunPCA(object = d, npcs = 50, features = VariableFeatures(object = d))
##
## Save file to new folder
```

```
# save(d, file = "case_study/d.RData")
# # matrix A: the main input of scBubbletree
# GBM_A <- d@reductions$pca@cell.embeddings[, 1:15]
#
# save(A, file = "GBM_data/GBM_A.RData")
# save(d@meta.data, file = "GBM_data/GBM_meta.RData")
```

To run this vignette we need to load a few packages:

```
library(scBubbletree)
library(ggplot2)
library(ggtree)
library(patchwork)
```

But first, load the PCA matrix, meta-data, and the matrix with normalized gene expressions:

```
# Load the data
# pca
load("GBM_data/GBM_A.RData")
# meta
load("GBM_data/GBM_meta.RData")
# gene expressions
load("GBM_data/GBM_exp.RData")
```

Step 1: determining the clustering resolution r for Louvain clustering

One important objective in scRNA-seq data analysis is to identify groups (clusters) of transcriptionally similar cells by clustering. *scBubbletree* offers several clustering methods (Louvain, Leiden, k-means, etc.), most of which are also implemented in *Seurat*. Here we will use the original Louvain method.

To do clustering we need two sets of inputs:

1. the PCA matrix
2. clustering resolution r

How to determine r ?

To determine the clustering resolution, we computed the gap statistic for the Louvain method with resolution parameter $r = 10^R$, where R was initialized using a sequence of values from -4 to 1 in steps of 0.1. At each resolution we recorded the number k' of identified communities.

```
rs <- 10^seq(from = -4, to = 1, by = 0.1)
```

Next, we run the function `get_r`:

```
data_r <- get_r(B_gap = 20, cores = 20, rs = rs,
               x = GBM_A, algorithm = "original")
save(data_r, file = "GBM_data/data_r.RData")
```

How does the gap curve behave as a function of r ?

The gap increased rapidly between $k' = 1$ and $k' = 13$, followed by a dip at $k' = 14$ to a value below 4. Between $k' = 13$ and $k' = 25$, the gap increased gradually by only about 0.15, even though 12 communities (about two-fold more) were added in this interval. This suggested that there is little benefit in using values of r larger than 0.158 ($k' = 13$). At higher k' we observed a slower and approximately linear increase in the gap.

The value of $k' = 13$ is close to the number of clusters identified in the publication by Schmassmann et al., 2023, which reports between $k = 10$ and $k = 22$ clusters of transcriptionally similar cells.

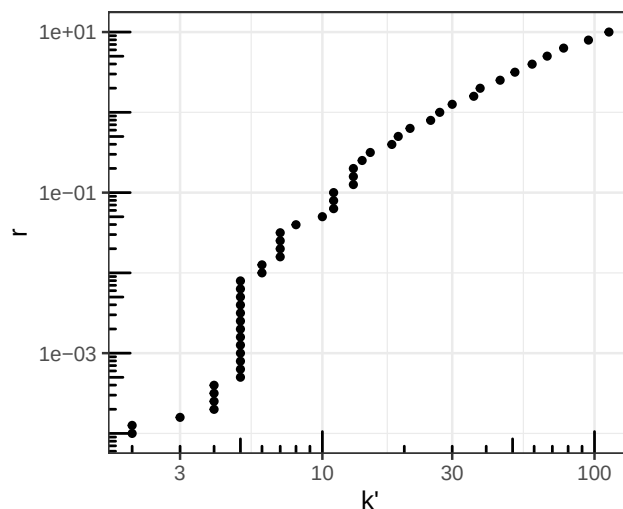
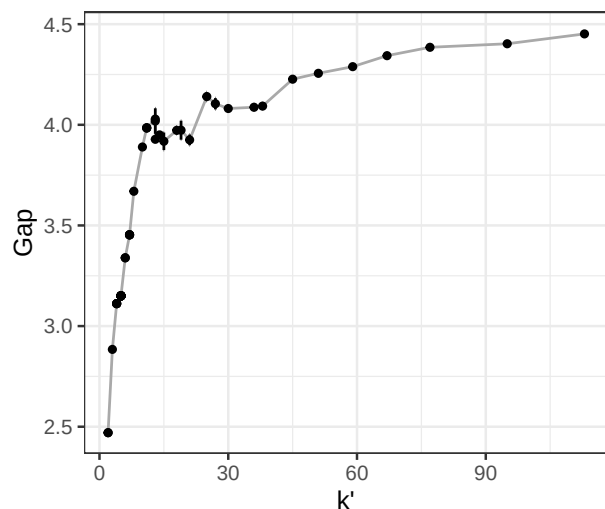
Clustering of cells was performed with hierarchical clustering on the Euclidean distances between cells (with Ward's criterion to minimize the total variance within each cluster Murtagh and Legendre, 2014; package cluster version 2.1.0). The number of clusters used for following analyses was identified by applying a dynamic tree cut (package dynamicTreeCut, version 1.63-1) (Langfelder et al., 2008), resulting in 10 with argument deepSplit set to 1, or 22 clusters with argument deepSplit set to 2.

To find e.g. rare tumor cells, one can increase the resolution to e.g. $r = 0.79$ ($k' = 25$) or $r = 2.51$ ($k' = 45$). Note, that scBubbletree's clustering workflow is transparent. It demands from the user to think and evaluate different clustering resolutions and thereby understand the data. The only cost is the additional computational time required to run `get_r` or `get_k`.

```
g1 <- ggplot(data_r$gap_stats_summary)+
  geom_line(aes(y = gap_mean, x = k), col = "darkgray")+
  geom_point(aes(y = gap_mean, x = k), size = 1, col = "black")+
  geom_errorbar(aes(y = gap_mean, x = k, ymin = L95, ymax = H95), width = 0.1)+
  xlab(label = "k")+
  ylab(label = "Gap")+
  theme_bw(base_size = 10)

g2 <- ggplot(data_r$gap_stats_summary)+
  geom_point(aes(x = k, y = r), size = 1, col = "black")+
  ylab(label = "r")+
  xlab(label = "k")+
  scale_x_log10()+
  scale_y_log10()+
  annotation_logticks(base = 10, sides = "lb")+
  theme_bw(base_size = 10)

g1|g2+plot_annotation(tag_levels = 'A')
```



Step 2-3: clustering and hierarchical bubble grouping → bubbletree

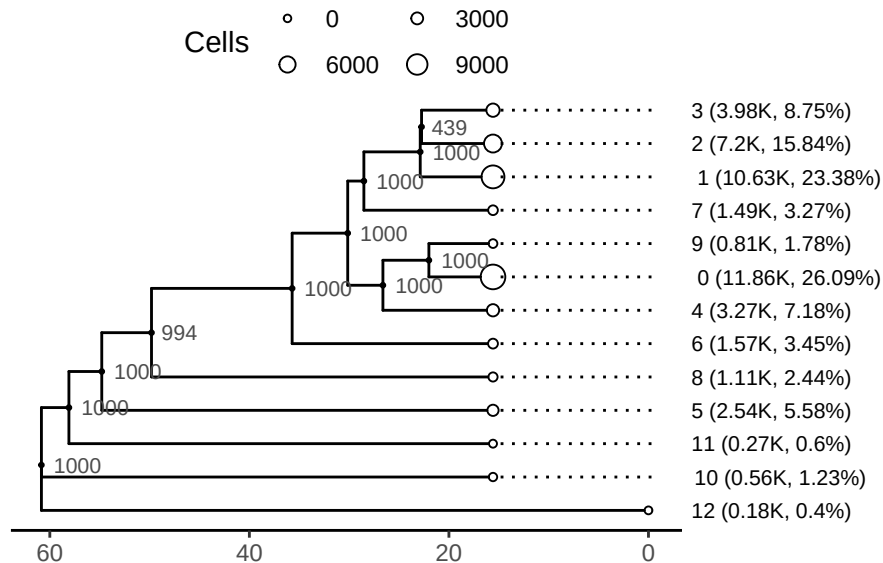
For clustering of dataset GBM we will use the original Louvain method and a resolution parameter $r = 0.158$ ($k' = 13$). First, we will construct an SNN graph based on $A^{45,466 \times 15}$, where each vertex (cell) in the SNN graph is connected to its 50 nearest neighbors. Clustering will be performed with 20 random starts and up to 100 iterations in each run. The clusters will be organized in a bubbletree using hierarchical clustering with average linkage. For this we will use $B = 1,000$ bootstrap iterations and draw samples with up to $N_{eff} = 100$ cells from each cluster to compute inter-cluster distances:

```
# graph-based clustering -> bubbletree
btd <- get_bubbletree_graph(x = GBM_A,
  r = 0.158,
  algorithm = "original",
  hclust_method = "average",
  hclust_distance = "euclidean",
  cores = 5,
  B = 1000,
  show_simple_count = TRUE,
  N_eff = 100)
```

Step 4: bubbletree visualization and annotation

Lets inspect the bubbletree

```
btd$tree
```



The resulting bubbletree has 13 bubbles with sizes ranging from about 11,800 cells (26%, cluster 0) down to 184 cells (0.4%, cluster 12). Nearly all branches in the bubbletree are completely robust. The branch between bubbles 2 and 3 has low branch support as it is found in only 457 out of 1,000 bootstrap dendrograms.

Annotating the bubbletree with cell features

For biological interpretation of the bubbletree structure, we will annotate the bubbletree with cluster-specific summaries of different cell features. Here, we will use the function `get_cat_tiles` to generate heatmaps for each of the following categorical features, which are provided as part of the meta-data:

1. coarse-grained cell type labels

2. fine-grained cell type labels
3. cell phases
4. sample labels

We will use the input parameter `integrate_vertical = FALSE`. This means that we will show the composition of each bubble (y-axis) with respect to the categories (labels; x-axis) of each feature, i.e. the within-bubble relative frequency of the different labels.

To visualize numeric features, such as mean gene expressions in each bubble, we will use the functions `get_num_tiles`, `get_num_violins`, and `get_num_cell_tiles`.

What is the cell type composition of the bubbles?

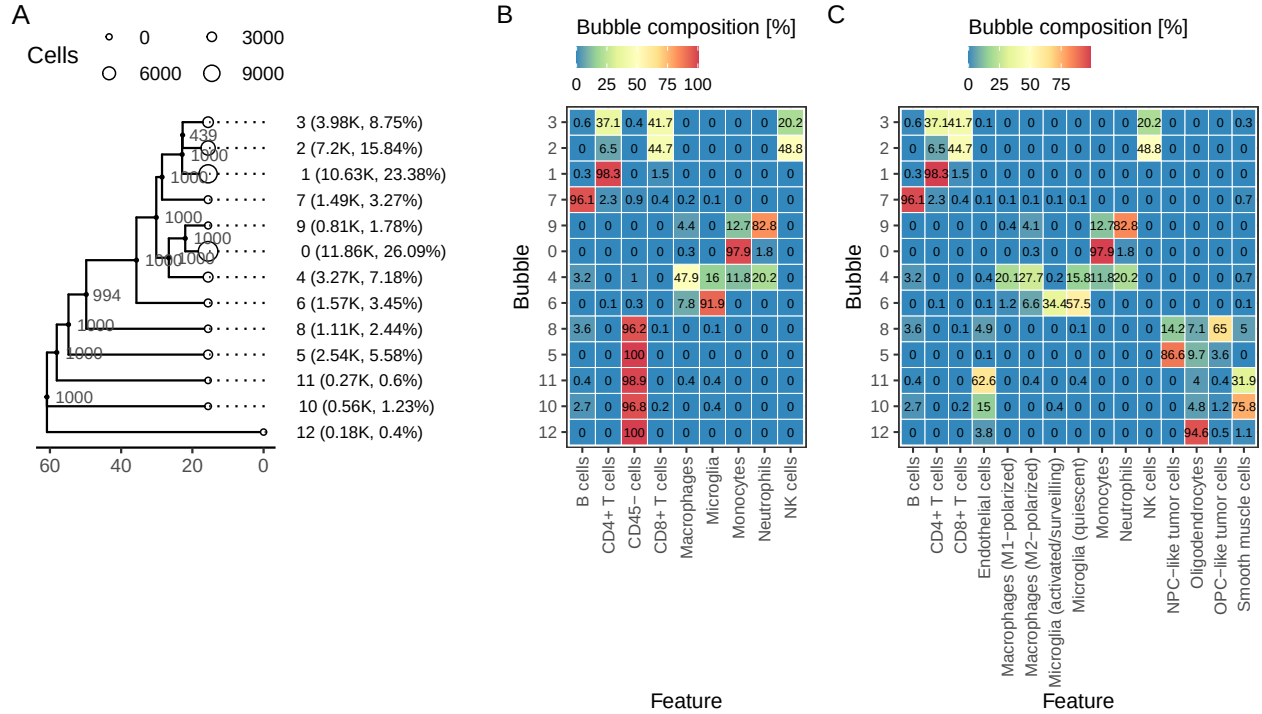
To determine which cell types are enriched in each bubble, we will visualize the within-bubble relative frequencies of fine- and coarse-grained cell type labels (heatmaps in Figs. B and C, below). To this end, we will run the function `get_cat_tiles` using as input two categorical features (two columns in `GBM_meta`: `CellAnnotation` and `CellAnnotationFine`), which provide the predicted cell type labels of each cell at two levels of resolution: coarse- grained and fine-grained, respectively.

```
# cell type annotations
g_c <- get_cat_tiles(btd = btd,
                    f = GBM_meta$CellAnnotation,
                    round_digits = 1,
                    tile_text_size = 2,
                    integrate_vertical = FALSE)

# cell type 'fine' annotations
g_cf <- get_cat_tiles(btd = btd,
                     f = GBM_meta$CellAnnotationFine,
                     round_digits = 1,
                     tile_text_size = 2,
                     integrate_vertical = FALSE)

g_ct <- (btd$tree|g_c$plot|g_cf$plot)+
  plot_layout(widths = c(1, 1, 1.5))+
  plot_annotation(tag_levels = 'A')

g_ct
```



The analysis shows that more than 95% of the cells in bubbles 8, 5, 11, 10, and 12 were CD45- (non-hematopoietic cells), while the remaining bubbles are enriched with immune cells, consistent with the results reported in Schmassmann et al., 2023.

The bubbletree conserves global transcriptional distances between bubbles, grouping them into a PBMC clade with T-cells (bubbles 3, 2, and 1), B-cells (bubble 7), NK-cells (bubbles 3 and 2), monocytes (bubbles 9, 0, and 4), and neutrophils (bubbles 9 and 4). Additionally, it identifies multiple non-PBMC populations including microglia (bubble 6), macrophages (bubble 4), tumor cells (bubbles 5 and 8), and other smaller cell populations.

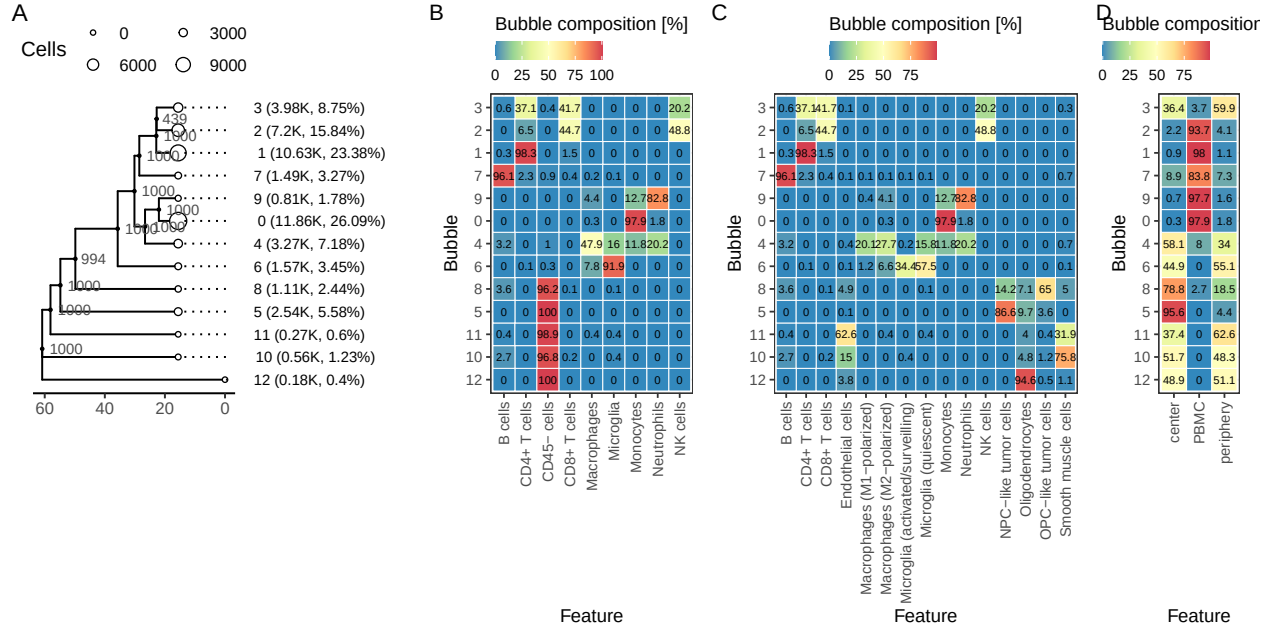
What is the tissue composition of the bubbles?

Dataset GBM contains samples from three tissues: PBMC, tumor center, and peripheral infiltration zone. Using tissue as categorical cell features, we will next visualize the relative frequency of cells from a specific tissue within the different bubbles (heatmap in Fig. D).

```
# tissue
g_t <- get_cat_tiles(btd = btd,
  f = GBM_meta$tissue,
  round_digits = 1,
  tile_text_size = 2,
  integrate_vertical = FALSE)

g3 <- (g_ct|g_t$plot)+
  plot_layout(widths = c(1, 1, 1.5, 0.4))+
  plot_annotation(tag_levels = 'A')

g3
```



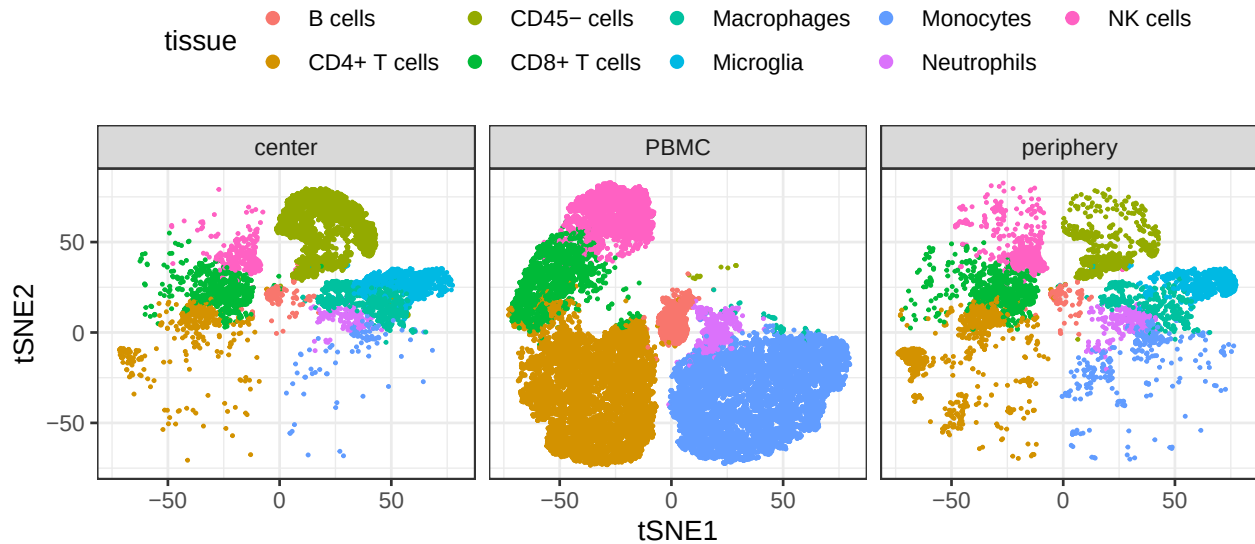
We observe that many bubbles enriched with PBMC cell types had high relative frequency of cells from PBMC tissue, except for bubble 3, which contains a mixture of cells from the tumor center and peripheral infiltration zone tissues. Hence, bubble 3 may contain tumor-infiltrating lymphocytes. The remaining bubbles in the bubbletree contain a mixture of cells from tumor center and peripheral infiltration zone tissues. Meanwhile, bubbles 5 and 8 are mostly composed of tumor cells from tumor centers.

Lets compare the previous visual with a 2D t-SNE plot

We will plot the same 2D t-SNE visual reported by Schmassmann et al., 2023.

```
g4 <- ggplot(data = GBM_meta)+
  facet_wrap(facets = ~tissue, nrow = 1)+
  geom_point(aes(x = tSNE1, y = tSNE2, col = CellAnnotation), size = 0.25)+
  theme_bw()+
  theme(legend.position = "top")+
  guides(colour = guide_legend(override.aes = list(size=2)))+
  scale_color_discrete(name = "tissue")
```

g4



Global distances between the clusters are **not** adequately conserved in the 2D t-SNE plot. Furthermore, the 2D t-SNE plot suffers from heavy overplotting, despite the fact that the cells are split into three panels for each tissue site.

Compositional analysis: are cells from different samples evenly mixed within bubbles?

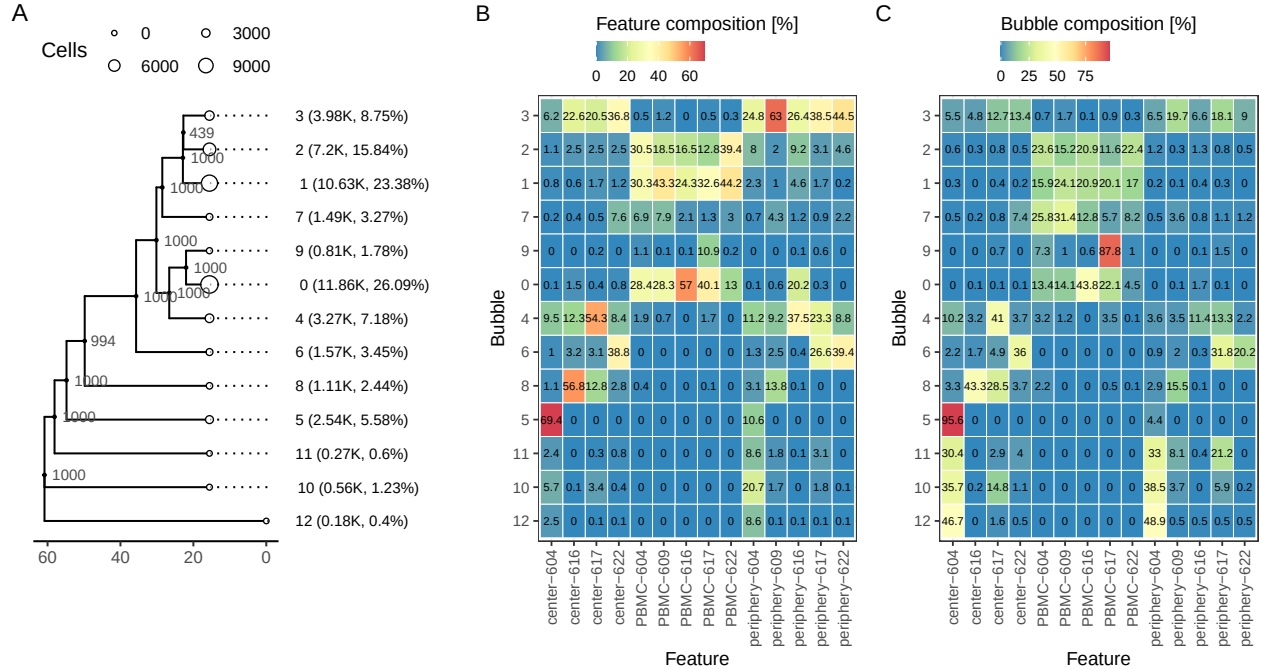
Dataset GBM contains samples from five patients and three tissues. Using tissue and patient as categorical cell features, we will visualize the relative frequency of cells from a specific tissue and patient across the different bubbles (heatmap in Fig. B, below) as well as within the bubbles (heatmap in Fig. C, below). This allows us to check for compositional biases, which is essential to e.g. identify novel cell types (e.g. tumor cells), or to detect systematic biases (e.g. batch effects) in a subset of the samples.

```
g_p_v <- get_cat_tiles(btd = btd,
  f = paste0(GBM_meta$tissue, '-', GBM_meta$patient),
  round_digits = 1,
  tile_text_size = 2,
  integrate_vertical = TRUE)

g_p_h <- get_cat_tiles(btd = btd,
  f = paste0(GBM_meta$tissue, '-', GBM_meta$patient),
  round_digits = 1,
  tile_text_size = 2,
  integrate_vertical = FALSE)

g5 <- (btd$tree|g_p_v$plot|g_p_h$plot)+
  plot_layout(widths = c(1, 1.3, 1.3))+
  plot_annotation(tag_levels = 'A')

g5
```



In the next, we describe the key compositional biases:

- First, all PBMC samples (middle columns in Fig. B, above) have similar relative abundances across the bubbles. In contrast, the samples from tumor center and peripheral infiltration zone are heterogeneous in terms of their relative abundances across the bubbles.
- Second, all bubble 5 cells come from patient 604 (95.6% from center-604 and 4.4% periphery-604) (Fig C, above), and 69.4% of the cells in sample center-604 are assigned to bubble 5 (NPC-like tumor cells)
- Third, the majority of cells in bubbles 11, 10, and 12 came from patients 604 and 617.

In summary, we observed high degree of GBM heterogeneity between individuals. Yet, despite these biases, from Schmassmann et al., 2023, we get the following remarks about the degree of overlap of cells from the different patients across cell types (clusters):

As we observed a good overlap of cells across patients for most of the dataset (see Methods; Figure 1-figure supplement 2B-F), we chose not to perform any correction for patient-specific effects

A quantitative test of overlap of cells across patients was made using the CellMixS package (Lütge et al., 2021) (version 1.12.0) developed to quantify the effectiveness of batch correction methods. We used the cell-specific mixing (CMS) score, which highlighted a very good overlap across cells from different patients in the lymphoid compartment and for the monocytes. The myeloid compartment displayed a slightly elevated patient-specific structure, while it was most pronounced for the CD45 negative subset.

Quality control with *scBubbletree*

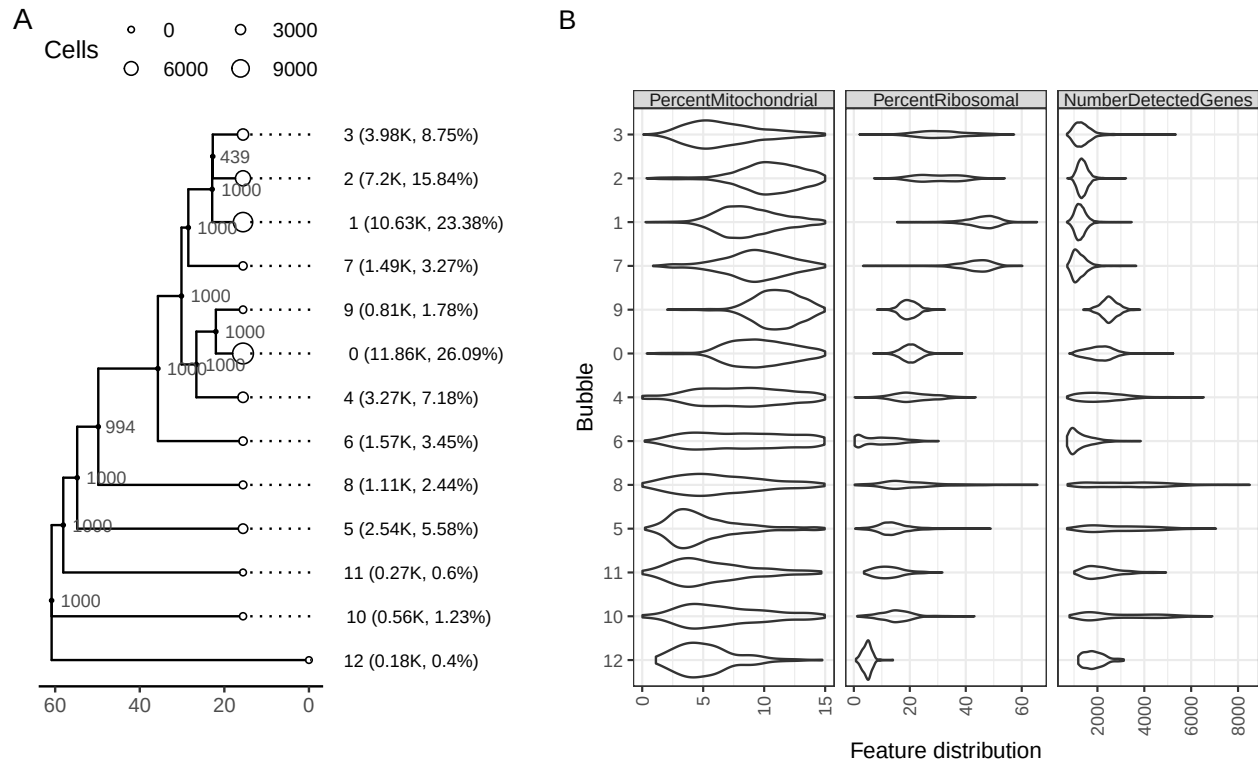
Finally, nearly every scRNA-seq data analysis begins with quality control (QC). *scBubbletree* allows you to explore QC metrics commonly used by the community:

1. the number of unique genes detected in each cell
2. total number of molecules detected within a cell
3. the percentage of reads that map to the mitochondrial genome

Furthermore, any cell feature (numeric or categorical) that can be used for QC. These can be attached to the bubbletree to identify bubbles enriched with low-quality cells.

```
g_qc <- get_num_violins(btd = btd,
  f = as.matrix(GBM_meta[, c("PercentMitochondrial",
    "PercentRibosomal",
    "NumberDetectedGenes")]))

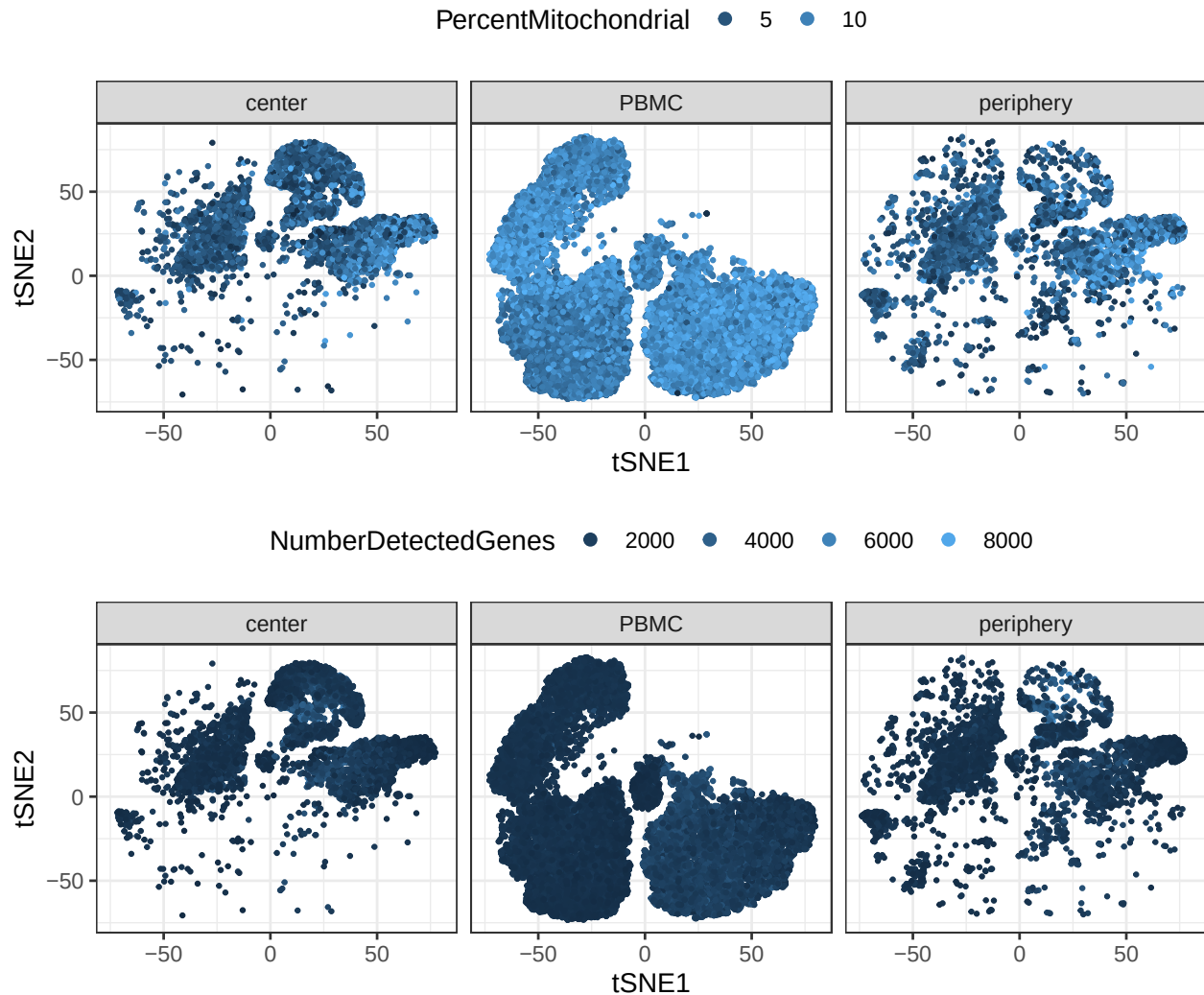
(btd$tree|g_qc$plot)+
  plot_layout(widths = c(1, 2.2))+
  plot_annotation(tag_levels = 'A')
```



According to Schmassmann et al., 2023, low quality cells have been filtered from GBM data before the data was archived. Hence, we did not expect to see bubbles that are enriched with low-quality cells.

Nevertheless, we want to remind the reader that scBubbletree can be used for QC. In fact, we argue that the above visual is simpler and more informative than the one used by Schmassmann et al., 2023 which suffers from overplotting (presented as Figure 1-figure supplement 1 C-D; the quality feature PercentRibosomal is not visualized in the publication). We have replicated their visual below:

```
(ggplot(data = GBM_meta)+
  facet_wrap(facets = ~tissue, nrow = 1)+
  geom_point(aes(x = tSNE1, y = tSNE2, col = PercentMitochondrial), size=0.5)+
  theme_bw()+
  theme(legend.position = "top")+
  guides(colour = guide_legend(override.aes = list(size=2))))/
(ggplot(data = GBM_meta)+
  facet_wrap(facets = ~tissue, nrow = 1)+
  geom_point(aes(x = tSNE1, y = tSNE2, col = NumberDetectedGenes), size=0.5)+
  theme_bw()+
  theme(legend.position = "top")+
  guides(colour = guide_legend(override.aes = list(size=2))))
```



Other cell features: cell phase and gene expression

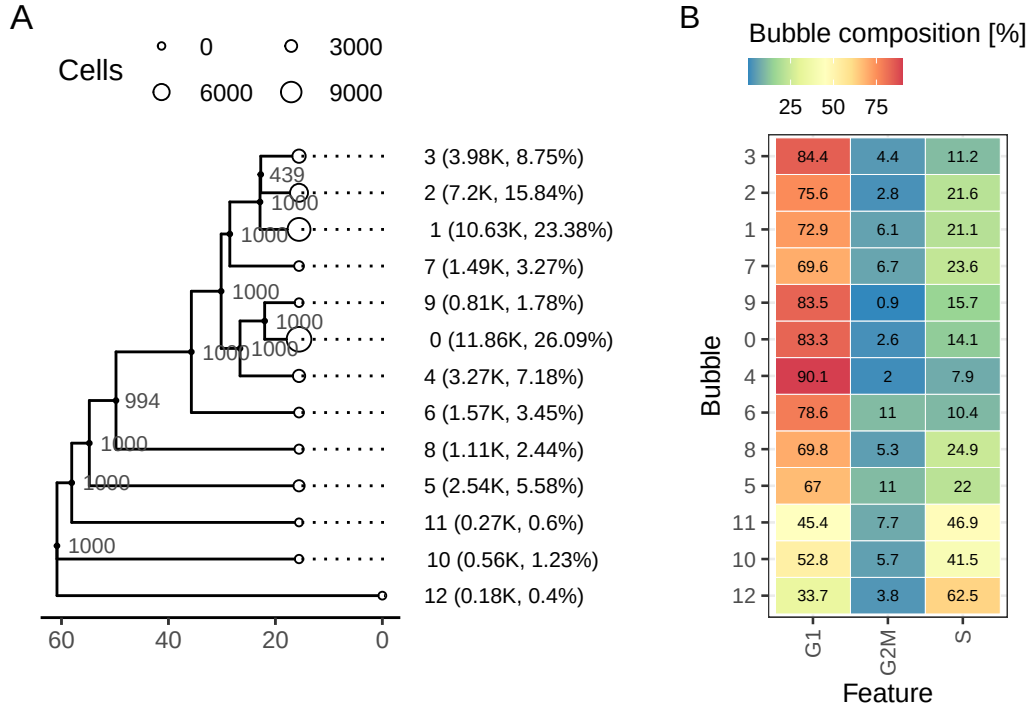
There is no limit to the number of type of cell features that can be used to annotate the bubbletree. In the next, we show how to ‘attach’ summaries of predicted cell phase labels and gene expressions to the bubbles.

What is the dominant cell phase in the bubbles?

Using cell phase as categorical cell features, we visualized the relative frequency of cells with a cell phase G1, G2M, and S, within the different bubbles.

```
# cell cycle phase
g_p <- get_cat_tiles(btd = btd,
                    f = GBM_meta$CellCyclePhase,
                    round_digits = 1,
                    tile_text_size = 2,
                    integrate_vertical = FALSE)

(btd$tree|g_p$plot)+
  plot_layout(widths = c(1.5, 1))+
  plot_annotation(tag_levels = 'A')
```



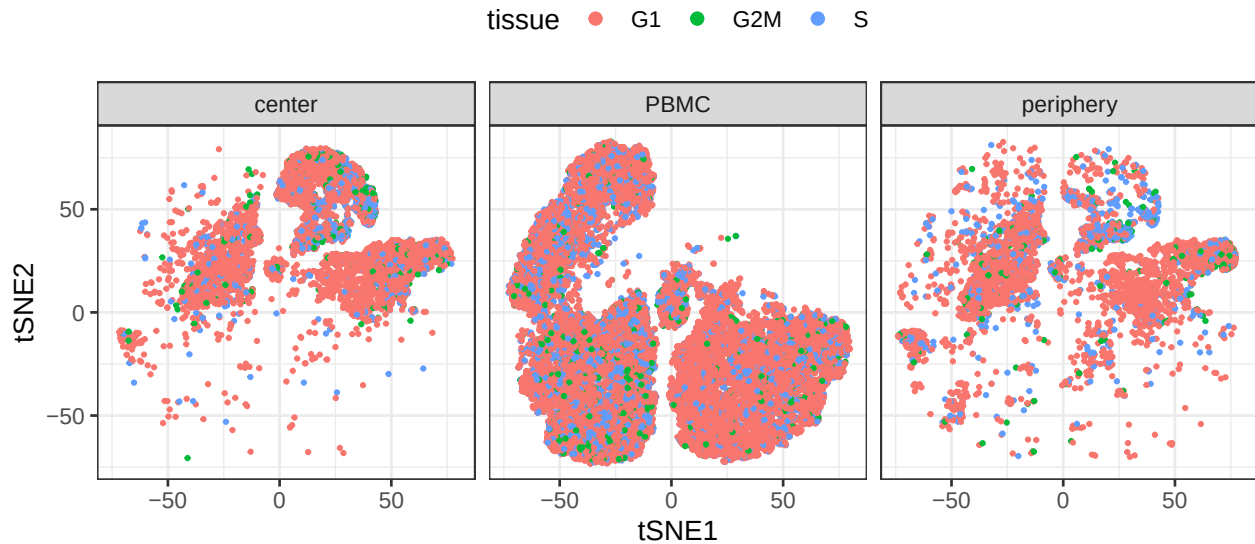
According to the heatmap, the dominant cell phases vary from bubble to bubble. The cells in the bubbles containing immune cells are predominantly in G1 phase, i.e. >73% of the cells in each of the bubbles 6, 4, 0, 9, 7, 1, 2 and 3 are in this phase. This makes sense from a biological point of view, i.e. most immune cells are likely resting, and in their quiescent state they do not need to proliferate, thus they remain in the G0/G1 phase until activated.

In contrast to this, tumor cells, are more likely to be found in S and G2/ M phases due to their constant division. This is exactly what we observe, i.e. we see higher fraction of S (synthesis phase) phase for cells in the bubbles containing tumor cell.

The quantitative summaries encoded in panel B makes the visual generated by scBubbletree more informative than the complementary t-SNE visual (see below), in which cells (dots) are color coded according to their cell phases (presented in Schmassmann et al., 2023 as Figure 1-figure supplement 1 E).

```
g7 <- ggplot(data = GBM_meta)+
  facet_wrap(facets = ~tissue, nrow = 1)+
  geom_point(aes(x = tSNE1, y = tSNE2, col = CellCyclePhase), size = 0.5)+
  theme_bw()+
  theme(legend.position = "top")+
  guides(colour = guide_legend(override.aes = list(size=2)))+
  scale_color_discrete(name = "tissue")
```

g7



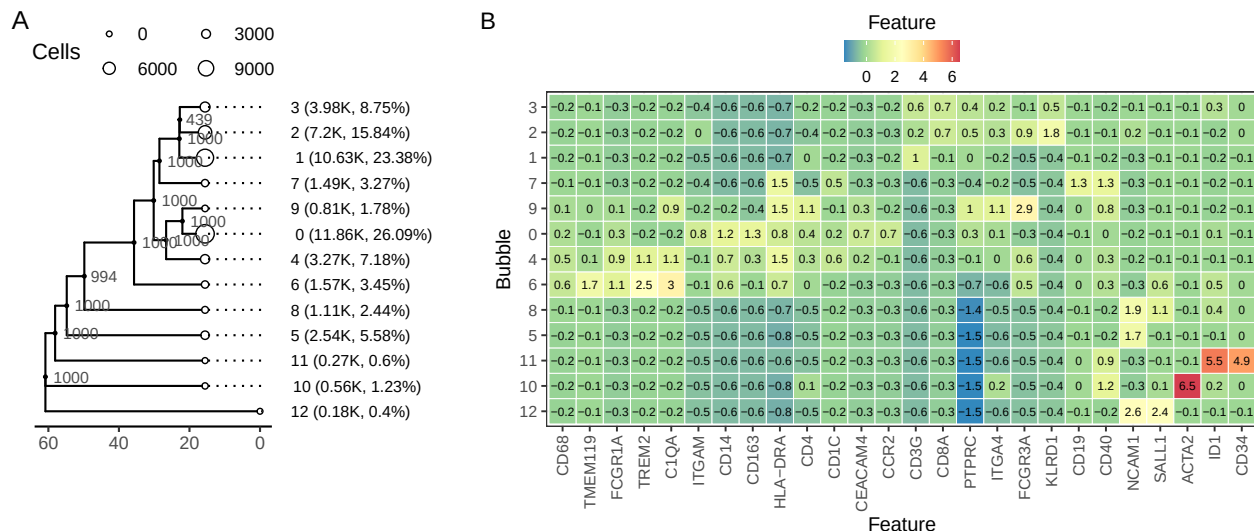
What is the average expression of marker genes in each bubble?

Next, we visualized the average expression values of 26 marker genes in each bubble (panel C). This is similar to figure 2B in Schmassmann et al., 2023. Numeric features, such as gene expressions, can be visualized with `scBubbletree`. For this we can use the functions `get_num_tiles` or `get_num_violins`. To generate panel C we used `get_num_tiles`.

```
# gene expression
g_e <- get_num_tiles(btd = btd,
                    fs = GBM_exp,
                    summary_function = "mean",
                    round_digits = 1,
                    tile_text_size = 2)

# gene expression
g_ce <- (get_num_cell_tiles(btd = btd, f = GBM_exp[,1], feature_name = "CD68"))$plot |
  (get_num_cell_tiles(btd = btd, f = GBM_exp[,2], feature_name = "TMEM119"))$plot |
  (get_num_cell_tiles(btd = btd, f = GBM_exp[,18], feature_name = "FCGR3A"))$plot |
  (get_num_cell_tiles(btd = btd, f = GBM_exp[,15], feature = "CD8A"))$plot

(btd$tree|g_e$plot)+
  plot_layout(widths = c(1, 3))+
  plot_annotation(tag_levels = 'A')
```



We can evaluate heatmap B to double-check whether the cell type predictions are consistent with the gene expression of cell type characteristic markers.

Here are few examples of this analysis:

- CD14 is a marker of monocytes. CD14 has high average expression in cells from bubble 0 (≈ 1.2). We know from the cell type annotation visual shown earlier that bubble 0 is enriched with monocytes.
- NCAM1 is a marker of different tumors. NCAM1 has high average expression in cells from the tumor cell enriched bubbles 5 (≈ 1.7) and 8 (≈ 1.9).

What is the expression of individual cells?

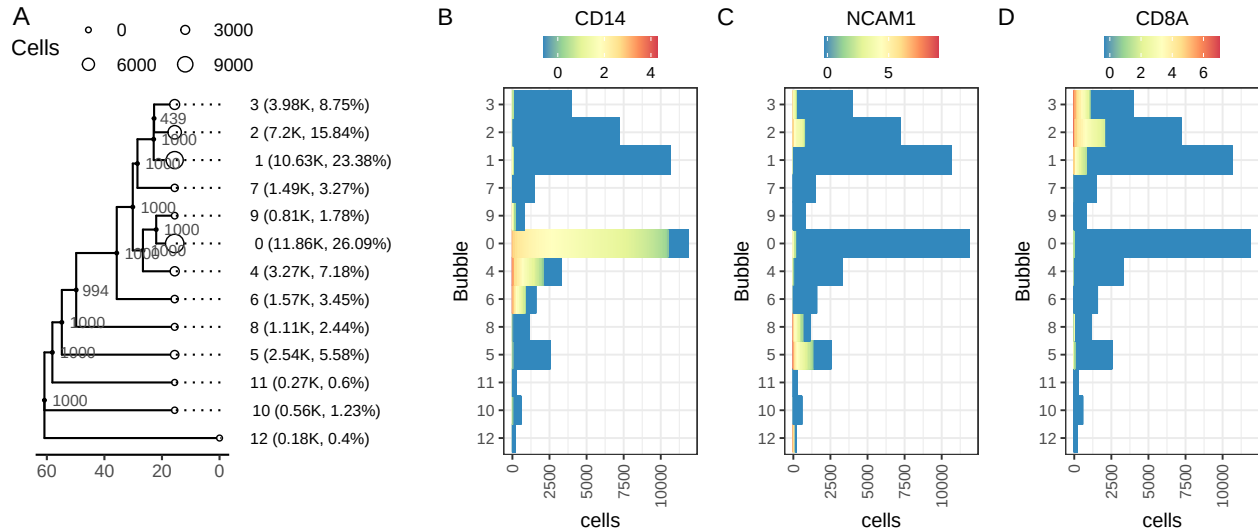
To visualize numeric features of individual cells we used the function `get_num_cell_tiles`, which visualizes numeric attributes of individual cells in each bubble using heatmaps, with the cells ordered by the magnitude of their numeric attributes. Lets do this for 3 marker genes: CD14, NCAM1 and CD8A. We see comparable results as in the heatmap generated by `get_num_tiles`.

```
# gene expression
g_cd14 <- get_num_cell_tiles(btd = btd,
                             f = GBM_exp[, c("CD14")],
                             feature_name = "CD14")

g_ncam1 <- get_num_cell_tiles(btd = btd,
                              f = GBM_exp[, c("NCAM1")],
                              feature_name = "NCAM1")

g_cd8a <- get_num_cell_tiles(btd = btd,
                             f = GBM_exp[, c("CD8A")],
                             feature_name = "CD8A")

(btd$tree|g_cd14$plot|g_ncam1$plot|g_cd8a$plot)+
  plot_layout(widths = c(1, 1, 1, 1))+
  plot_annotation(tag_levels = 'A')
```



Summary

scBubbletree promotes simple and transparent visual exploration of scRNA-seq. It is **not a black-box approach** and the user is encouraged to explore the data with different values of k and r or custom clustering solutions. Attaching cell features to the bubbletree is necessary for biological interpretation of the individual bubbles and their relationships which are described by the bubbletree.

Session Info

```
sessionInfo()
FALSE R version 4.4.1 (2024-06-14)
FALSE Platform: x86_64-pc-linux-gnu
FALSE Running under: Ubuntu 24.04 LTS
FALSE
FALSE Matrix products: default
FALSE BLAS: /usr/lib/x86_64-linux-gnu/atlas/libblas.so.3.10.3
FALSE LAPACK: /usr/lib/x86_64-linux-gnu/atlas/liblapack.so.3.10.3; LAPACK version 3.11.0
FALSE
FALSE locale:
FALSE [1] LC_CTYPE=en_US.UTF-8 LC_NUMERIC=C
FALSE [3] LC_TIME=en_US.UTF-8 LC_COLLATE=en_US.UTF-8
FALSE [5] LC_MONETARY=en_US.UTF-8 LC_MESSAGES=en_US.UTF-8
FALSE [7] LC_PAPER=en_US.UTF-8 LC_NAME=C
FALSE [9] LC_ADDRESS=C LC_TELEPHONE=C
FALSE [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
FALSE
FALSE time zone: Europe/Berlin
FALSE tzcode source: system (glibc)
FALSE
FALSE attached base packages:
FALSE [1] stats graphics grDevices utils datasets methods base
FALSE
FALSE other attached packages:
FALSE [1] patchwork_1.2.0 ggtree_3.12.0 ggplot2_3.4.4
FALSE [4] scBubbletree_1.7.20 BiocStyle_2.32.0
```

```

FALSE
FALSE loaded via a namespace (and not attached):
FALSE [1] RColorBrewer_1.1-3      rstudioapi_0.15.0      jsonlite_1.8.8
FALSE [4] magrittr_2.0.3          spatstat.utils_3.0-5   farver_2.1.1
FALSE [7] rmarkdown_2.25          fs_1.6.3               vctr_0.6.5
FALSE [10] ROCR_1.0-11             memoise_2.0.1          spatstat.explore_3.3-1
FALSE [13] htmltools_0.5.7         gridGraphics_0.5-1     sctransform_0.4.1
FALSE [16] parallelly_1.37.1       KernSmooth_2.23-24     htmlwidgets_1.6.4
FALSE [19] ica_1.0-3               plyr_1.8.9             plotly_4.10.4
FALSE [22] zoo_1.8-12              cachem_1.0.8           igraph_1.6.0
FALSE [25] mime_0.12               lifecycle_1.0.4        pkgconfig_2.0.3
FALSE [28] Matrix_1.7-0            R6_2.5.1               fastmap_1.1.1
FALSE [31] fitdistrplus_1.2-1      future_1.33.1          shiny_1.8.0
FALSE [34] digest_0.6.34           aplot_0.2.3            colorspace_2.1-0
FALSE [37] Seurat_5.1.0            tensor_1.5             RSpectra_0.16-1
FALSE [40] irlba_2.3.5.1          labeling_0.4.3         progressr_0.14.0
FALSE [43] fansi_1.0.5             spatstat.sparse_3.1-0  httr_1.4.7
FALSE [46] polyclip_1.10-6         abind_1.4-5            compiler_4.4.1
FALSE [49] proxy_0.4-27            withr_2.5.0            BiocParallel_1.38.0
FALSE [52] fastDummies_1.7.3       MASS_7.3-61           tools_4.4.1
FALSE [55] lmtest_0.9-40           ape_5.7-1             httpuv_1.6.14
FALSE [58] future.apply_1.11.1     goftest_1.2-3          glue_1.7.0
FALSE [61] nlme_3.1-165           promises_1.2.1         grid_4.4.1
FALSE [64] Rtsne_0.17             cluster_2.1.6          reshape2_1.4.4
FALSE [67] generics_0.1.3         gtable_0.3.4          spatstat.data_3.1-2
FALSE [70] tidyr_1.3.1            data.table_1.14.10     sp_2.1-2
FALSE [73] utf8_1.2.4             spatstat.geom_3.3-2    RcppAnnoy_0.0.22
FALSE [76] ggrepel_0.9.5          RANN_2.6.1            pillar_1.9.0
FALSE [79] stringr_1.5.1          yulab.utils_0.1.5     spam_2.10-0
FALSE [82] RcppHNSW_0.6.0         later_1.3.2           splines_4.4.1
FALSE [85] dplyr_1.1.4            treeio_1.28.0         lattice_0.22-6
FALSE [88] survival_3.7-0         deldir_2.0-4          tidyselect_1.2.0
FALSE [91] miniUI_0.1.1.1         pbapply_1.7-2         knitr_1.45
FALSE [94] gridExtra_2.3          scattermore_1.2        xfun_0.41
FALSE [97] matrixStats_1.2.0      stringi_1.8.3         lazyeval_0.2.2
FALSE [100] ggfun_0.1.5            yaml_2.3.8            evaluate_0.23
FALSE [103] codetools_0.2-20       tibble_3.2.1          BiocManager_1.30.22
FALSE [106] ggplotify_0.1.2        cli_3.6.2             uwot_0.2.2
FALSE [109] xtable_1.8-4           reticulate_1.35.0     munsell_0.5.1
FALSE [112] Rcpp_1.0.12            globals_0.16.2        spatstat.random_3.3-1
FALSE [115] png_0.1-8             spatstat.univar_3.0-0  parallel_4.4.1
FALSE [118] ellipsis_0.3.2         dotCall64_1.1-1       listenr_0.9.1
FALSE [121] viridisLite_0.4.2     tidytree_0.4.6        scales_1.3.0
FALSE [124] ggribes_0.5.6          SeuratObject_5.0.2    leiden_0.4.3.1
FALSE [127] purrr_1.0.2           rlang_1.1.3           cowplot_1.1.3

```